



ZILLOW HOUSE SALES

*Supervise by: DR. Mohamed Abdelhafeez
Eng. Hager Nasser*

Member	Id	Role
Esraa Mostafa	22010321	Web Scraping
Ghada Elsayed	22010371	Data Preprocessing and EDA
Maya Mahmoud	22010204	Machine Learning

Zillow House Sales Data Analysis

1. Introduction

The real estate industry plays a critical role in global economic systems, influencing personal investments, corporate strategies, and public policy. Platforms like Zillow have transformed how data is shared and accessed, offering real-time information about properties, pricing, and market trends. This abundance of information presents an opportunity to leverage data science tools to derive actionable insights for better decision-making.

This project focuses on analyzing Zillow's Nebraska property listings by employing web scraping, data cleaning, and machine learning methodologies. Web scraping was used to automate the extraction of data from Zillow's website. The raw data was then cleaned and transformed into a structured dataset, followed by applying machine learning models to predict housing prices based on property features.

2. Objectives:

1. To collect comprehensive real estate data from Zillow using automation.
2. To transform unstructured web data into a usable, clean format.
3. To derive insights about pricing trends and housing features using machine learning models.

This project demonstrates the significance of integrating web scraping and predictive analytics in tackling real-world challenges, showcasing how data-driven approaches can provide valuable insights in the real estate market.

apiWeb Scrng: Extracting Zillow's Data:

Web scraping is a critical step in this project, enabling the automated collection of real estate data from Zillow. The scraping process uses Python libraries to simulate a browser, retrieve HTML content, and parse structured information from the webpage. By automating this process, we eliminate the inefficiencies of manual data collection, paving the way for large-scale analysis.

1. Step 1: Sending HTTP Requests:

The process begins by using the requests library to send HTTP GET requests to the Zillow website.

```
import requests
```

```
BASE_URL = "https://www.zillow.com/ne"
```

```
HEADERS = {
```

```
    "User-Agent": 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36',
```

```
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8",}
```

```
response = requests.get(BASE_URL, headers=HEADERS)
```

- **Purpose:** Retrieve the raw HTML content of Zillow's Nebraska listings.
- **Challenges Addressed:**
 - **Anti-scraping:** Headers were included to mimic browser requests, reducing the likelihood of being blocked.
 - **Rate Limiting:** To avoid IP bans, a `time.sleep(3)` delay was added between requests for subsequent pages.

➤ Step 2: Navigating Pages

To scrape multiple pages, the script dynamically appends a page number to the URL after the first page.

```
url = BASE_URL if page == 1 else f"{BASE_URL}/{page}_p"
```

- Why? Zillow organizes property listings across several pages. Automating the navigation ensures comprehensive data collection.
- Limitations: Zillow restricts the number of accessible pages to approximately twenty-five due to anti-scraping measures.

➤ Step 3: Parsing HTML Content

The HTML content of each page is parsed using BeautifulSoup to locate the `<script>` tag containing structured data in JSON format.

```
from bs4 import BeautifulSoup
```

```
soup = BeautifulSoup(response.content, 'html.parser')
```

```
script_content = soup.find('script', id='__NEXT_DATA__')
```

- Why Target the `<script>` Tag?
 - Zillow embeds property data in a JSON object within the `<script>` tag for dynamic page rendering.

- Parsing this tag provides access to structured data without manually extracting each HTML element.
- Key Challenge: If the <script> tag is missing or malformed, the script gracefully exits with an error message.

➤ Step 4: Extracting JSON Data

The extracted **<script>** tag content is converted into a JSON object for easy data manipulation.

```
import json
```

```
json_content = script_content.string
```

```
data = json.loads(json_content)
```

- Why Convert to JSON? JSON is hierarchical and structured, making it easier to access specific attributes such as price, address, and property size.
- Insights Gained: This step simplifies extracting nested fields, reducing errors and improving data consistency.

➤ Step 5: Data Extraction

Relevant fields such as price, address, and house size are extracted from the JSON object and organized into a list of dictionaries.

- Understanding the JSON Structure
- When parsing the JSON data from the <script> tag (`__NEXT_DATA__`), we observed that Zillow uses a hierarchical structure to organize property details. Specifically:
 - The root key, **props**, contains metadata about the page and application state.
 - Within **props**, the key **pageProps** stores the state of the search results.
 - The specific property listings are nested under **searchPageState** -> `cat1` -> `searchResults` -> `listResults`.

➤ Step 6: Saving Data to CSV

The structured data is written to a CSV file for further analysis.

```
import csv
```

```
with open("zillow_scraped_data.csv", 'w', newline="", encoding='utf-8') as file:
```

```
    writer = csv.DictWriter(file, fieldnames=scraped_data[0].keys())
```

```
    writer.writeheader()
```

```
    writer.writerows(scraped_data)
```

- Why Save as CSV? CSV is a universally accepted format for storing structured data, compatible with tools like Excel and Pandas.

➤ Challenges Encountered

1. Anti-Scraping Measures:

Zillow limits the number of accessible pages to approximately twenty-five, which explains the error encountered on page twenty-five.

Solution: Delays between requests and headers mimicking browsers were used to extend scraping limits.

2. Dynamic Content:

Property details are loaded dynamically using JavaScript, requiring JSON parsing instead of traditional HTML scraping.

Solution: The script directly targets the <script> tag containing the pre-rendered JSON data.

3. Incomplete Data:

Some listings lacked fields like price or size, necessitating the use of default values ('N/A').

➤ Insights Gained from Web Scraping

1. **Efficiency:** Automating the scraping process saves hours compared to manual data collection.
2. **Scalability:** The script can be adapted to other Zillow regions or similar platforms with minimal adjustments.
3. **Limitations:** Anti-scraping measures restrict data collection beyond twenty-five pages, which may require additional strategies like rotating proxies for larger datasets.

3.Data Preprocessing

➤ 1. Introduction

The preprocessing phase is essential in preparing raw data for machine learning models. In this report, we describe the preprocessing steps applied to a housing dataset. The dataset includes various features of properties such as price, size, location, and more. The goal of preprocessing is to clean the data, handle missing values, outliers, and irrelevant features, and ensure that the data is in an appropriate format for analysis or further modeling.

➤ 2. Dataset Overview

The dataset consists of 984 entries, with 16 columns describing various aspects of the houses. These include information such as:

- **House URL:** The link to the online listing.
- **Photo URLs:** Links to images of the property.
- **Price:** The listing price of the house.
- **Full Address:** The address of the property.
- **City, State, ZIP Code:** Location details.
- **Number of Bedrooms, Number of Bathrooms:** Property details.
- **House Size (sqft), Lot Size:** Dimensions of the property.
- **House Type:** The category or style of the house (e.g., townhouse, condo).
- **Home Status:** The current status of the property (e.g., for sale, sold).
- **Latitude, Longitude:** Geographical coordinates.
- **Street Address:** The specific street address of the house.

> 3. Data Exploration

A- Data Structure

The dataset contains 16 columns with 984 entries. Some columns contain missing values, and a few columns include irrelevant data.

B- Handling Missing Data

The following columns had missing values:

- **PHOTO URLs:** 2 missing values.
- **NUMBER OF BATHROOMS:** 1 missing value.
- **HOUSE SIZE (sqft):** 2 missing values.

We applied the following preprocessing steps to handle these:

- **PHOTO URLs:** Rows with missing photo URLs were removed since properties without images might be unreal.
- **NUMBER OF BATHROOMS** and **HOUSE SIZE (sqft):** Missing values were replaced with the median (to avoid outlier's effect) value of the respective columns.

C- Removing Irrelevant Features

The following columns were deemed irrelevant and were removed from the dataset:

- **HOUSE URL**—> It has been removed because of it wasn't helpful in my study case

- **PHOTO URLs**—> It has been removed because of it wasn't helpful in my study case
- **STREET ADDRESS**—> The 'FULL ADDRESS' column includes the STREET ADDRESSES
- **STATE** (as it had the same value for all houses)
- **HOME STATUS** (as it had the same value for all houses)

We also added an **ID** column for better identification of each house.

python

> 4. Data Cleaning

A- Price Conversion

The **PRICE** column contained values with special characters (e.g., dollar signs and commas) and abbreviations (e.g., "K" for thousands). The following function was applied to clean and convert the price to a numerical format (in dollars):

```
def clean_price(price):
    price = price.replace('$', '').replace(',', '') # Remove $ and commas
    if 'K' in price: # Handle 'K' for thousands
        price = price.replace('K', '')
        return float(price) * 1000
    return float(price)

df['PRICE$'] = df['PRICE'].apply(clean_price)
```

B- Lot Size Conversion

The **LOT SIZE** column contained both acre and square foot measurements. We converted all values to square feet for consistency:

> 5. Outlier Detection

A- Visualizing Outliers

We used boxplots to visually identify outliers in the numerical columns

B- Identifying Outliers Using IQR

For the **PRICES** column, we calculated the Interquartile Range (IQR) and used it to identify outliers

CONCLUSION :WE FOUND OUT THE DIFFERENCE OF PRICES ISN'T AN ABNORMAL OUTLIERS SO WE DECIDED TO NOT REMOVE THEM

➤ **6. Exploratory Data Analysis (EDA)**

A- Geospatial Distribution

A scatter plot was created to visualize the distribution of houses based on their latitude and longitude

B- Correlation Analysis

A correlation matrix was generated to assess the relationships between numeric columns, and a heatmap was plotted to visualize the results

4. Machine Learning

This Machine learning technique is used to solve a regression problem (predict house prices) based on various features. The process involves several key steps: data preprocessing, feature selection and the application of a Random Forest Regressor model for making predictions. Additionally, a user-friendly interface for predictions is developed using Gradio, allowing for easy input and display of results.

1. Handling Data for the Model

- Replaces invalid or missing values in the LOT SIZE (sqft) column (e.g., "N/A" or empty strings) with 0 then converts the column to numeric for further processing.
- Filters rows with realistic thresholds for HOUSE SIZE (sqft), NUMBER OF BEDROOMS, and NUMBER OF BATHROOMS.
- Tried to Scale the numerical features using StandardScaler to normalize the data for better model performance but didn't see much of a difference.

2. Encoding for Categorical Features

- Converts the categorical column HOUSE TYPE into binary columns by **One hot encoding**

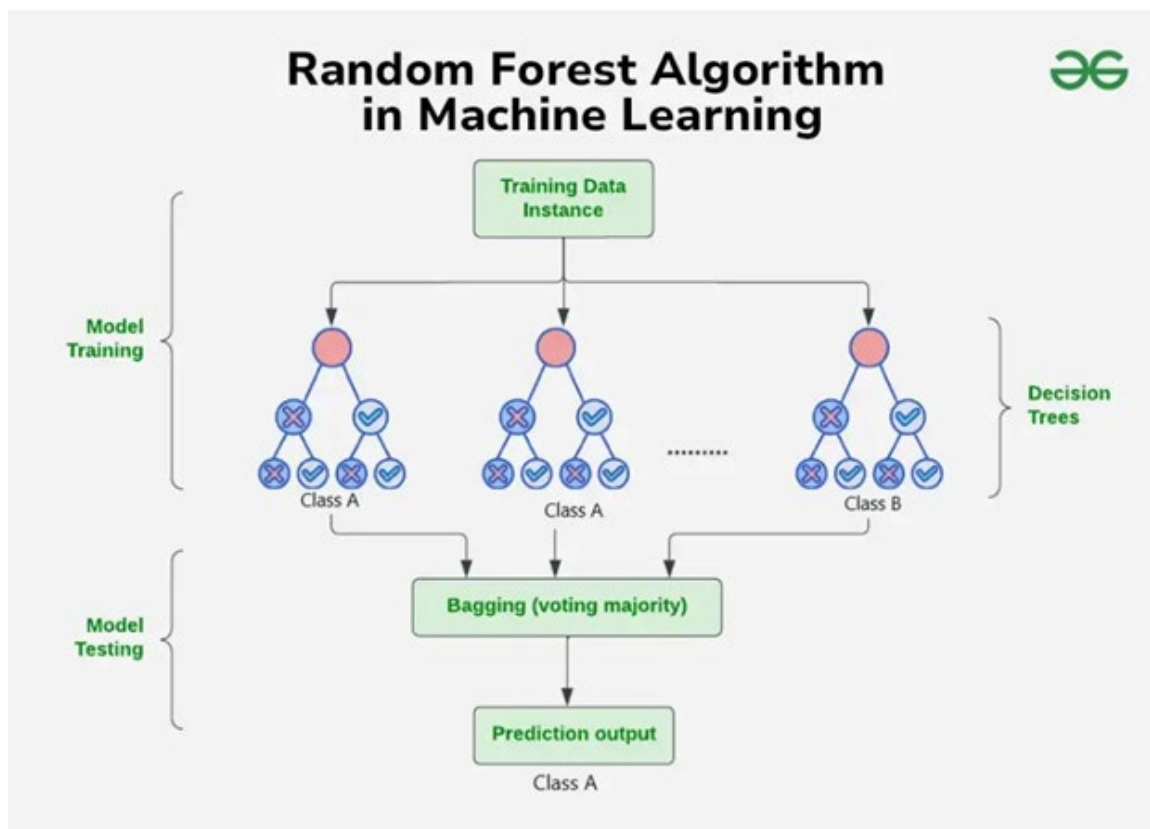
HOUSE TYPE_LOT	HOUSE TYPE_MANUFACTURED	HOUSE TYPE_MULTI FAMILY	HOUSE TYPE_SINGLE FAMILY	HOUSE TYPE_TOWNHOUSE
082.000000	082.000000	082.000000	082.000000	082.000000

- Encodes the CITY column into numerical values using **Label Encoding**.

3. Feature Selection

I experimented with several algorithms to determine the most effective one for my dataset, ultimately selecting the Random Forest Regressor due to its superior performance in handling the specific characteristics of my data

Random Forest is an ensemble learning technique that constructs multiple decision trees during training and merges their outputs to improve predictive accuracy and control overfitting. The core idea is that by combining the predictions from many individual trees, the overall model can achieve better performance than any single tree alone



- Extract the relative importance of each feature to understand their impact on the target variable

How Algorithm for Random Forest Work:

1. Step 1: Select random K data points from the training set.
2. Step 2: Build the decision trees associated with the selected data points (Subsets).
3. Step 3: Choose the number N for decision trees that you want to build.
4. Step 4: Repeat Step 1 and 2.
5. Step 5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

I selected the key features that would be used to predict prices, which included 'NUMBER OF BATHROOMS', 'HOUSE SIZE (sqft)', 'LOT SIZE (sqft)', 'CITY', 'ZIP CODE'

then separated these features into a variable called X, while the target variable, which is the house price, was stored in a variable called y.

To ensure that my model could be effectively trained and tested, I split the dataset into two parts: 80% of the data was used for training the model, and 20% was set aside for testing its performance. This division helps assess how well the model can predict prices based on new, unseen data.

4. Model Building and Evaluation

After initially training the Random Forest Regressor without any hyperparameter tuning, I obtained the following performance scores:

- **Random Forest Regressor Train R^2 Score:** 0.94 this score indicated overfitting
- **Random Forest Regressor Test R^2 Score:** 0.71
- **Mean Absolute Error (MAE):** 81,575.77
- **Mean Squared Error (MSE):** 28,726,350

These results indicated an overfitting on the training data but a noticeable drop in performance on the test data.

To improve the model's performance, I implemented hyperparameter tuning using Grid Search and did Cross validation because the original data set did not have enough records. I defined a parameter grid that included

- estimator: takes an estimator object, such as a classifier or a regression model.
- param_grid: takes a dictionary or a list of dictionaries. The dictionaries should be key-value pairs, where the key is the hyper-parameter and the value are the cases of hyper-parameter values to test.
- cv: takes an integer that determines the cross-validation strategy to apply. If None is passed, then 5 is used.
- scoring: takes a string or a callable. This represents the strategy to evaluate the performance of the test set.
- n_jobs: represents the number of jobs to run in parallel. Since this is a time-consuming process, running more jobs in parallel (if your computer can handle it) can speed up the process.
- verbose: determines how much information is displayed. Using a value of 1 displays the time for each run. 2 indicates that the score is also displayed. 3 indicates that the fold and candidate parameter are also displayed.

Using GridSearchCV, I performed a comprehensive search over these parameters with **5-fold cross-validation**. After tuning, I identified the best model and its parameters:

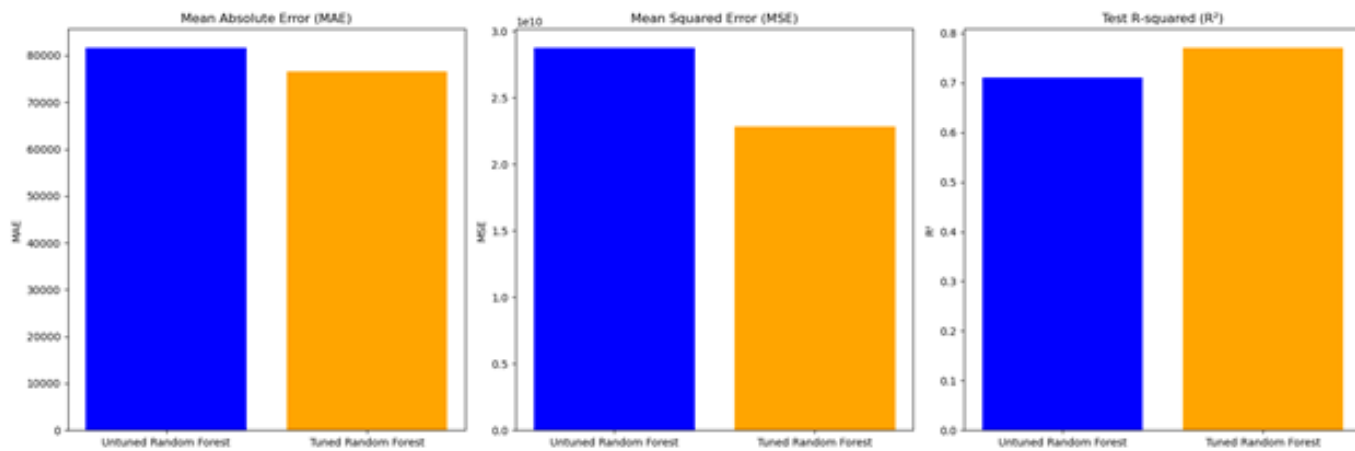
- **Best parameters found:**
 - max_depth: 10
 - max_features: 'sqrt'
 - min_samples_leaf: 1
 - min_samples_split: 5
 - n_estimators: 200

I then evaluated the tuned model on both the training and test sets and obtained the following results:

- **Tuned Random Forest Train R^2 : 0.89**
- **Tuned Random Forest Test R^2 : 0.77**
- **MAE: 76,515**
- **MSE: 22,852,427**

Additionally, I conducted another cross-validation to assess the stability of the model's performance:

● **Cross-Validation R^2 : 0.71 ± 0.04**



The tuning process improved accuracy on the test set and reduced errors compared to the initial model without tuning.

5. Model Deployment

The `predict_price` function is designed to help the user estimate house prices by collecting necessary information

First, it prompts the user to input key details such as the size of the house in square feet, the size of the lot in square feet, the number of bathrooms, the ZIP code, and the city where the house is located. the function organizes it into a structured format using a pandas DataFrame. This format makes it easier to process the data for predictions.

Next, the function encodes the city name into a numerical format using a pre-trained label encoder. This step is important because machine learning models require numerical input. If the entered city is not recognized, an error message is displayed, informing the user of valid city options. After preparing the data, the function uses a trained Random Forest Regressor model to predict the

House Price Predictor

House Size (sqft)	Predicted Price
1566	\$215,026.99
Lot Size (sqft)	Flag
9178	
Number of Bathrooms	
1	
Zip Code	
68506	
City	
Lincoln	
Clear	Submit

Use via API  · Built with Gradio 

house price based on the input features. Finally, it displays the predicted price in a clear and formatted manner for easy understanding.

Conclusion

- The Zillow House Data Analysis project demonstrates the integration of advanced data science techniques to solve real-world challenges in the real estate market. By combining web scraping, data cleaning, and machine learning, we successfully automated the process of collecting, structuring, and analyzing Zillow's Nebraska property listings.
- **Web Scraping** served as the foundation of this project, automating data collection from multiple pages of Zillow's website. Using Python libraries like requests, BeautifulSoup, and json, we overcame challenges such as anti-scraping mechanisms and dynamic content loading. By navigating Zillow's nested JSON structures, we efficiently extracted relevant fields such as prices, addresses, and property features. However, scraping limitations like page restrictions and inconsistent data required careful handling, ensuring the creation of a robust dataset for subsequent phases.
- **Data Cleaning** focused on transforming raw, unstructured data into a consistent format suitable for analysis. Missing values were imputed, prices were converted to numerical formats, and irrelevant or duplicate records were removed. This step was crucial in ensuring the accuracy and reliability of the dataset, eliminating outliers and inconsistencies that could skew the results.
- **Machine Learning** enabled predictive modeling to identify patterns and relationships between property attributes and prices. By leveraging features like house size, number of bedrooms, and location, we trained regression models to forecast property prices with significant accuracy.